"tests." When you write a test in JUnit or NUnit, it's difficult to avoid mentioning it somewhere. You use the `@Test` annotation, for example, or you make assertions about the result value. These conventions come from the mindset of writing unit tests once the code is complete, which, as you've seen, doesn't reflect how BDD and TDD work. Even if you try to adopt more BDD-style naming conventions, xUnit tests are still focused on *verification* rather than *specification*. In BDD, the idea is to use unit tests to implement low-level executable specifications, not to test code that has already been written.

Members of the Ruby community have traditionally been very early adopters of BDD practices and tools, and in 2005 RSpec, the first BDD unit-testing tool, was released (http://rspec.info). RSpec is fairly typical of a whole family of low-level BDD tools, so we'll look at RSpec first, and then see what other similar libraries exist for other languages.

WRITING LOW-LEVEL EXECUTABLE SPECIFICATIONS IN RUBY WITH RSPEC

The main idea behind RSpec is, in true BDD fashion, to think in terms of describing application behavior rather than verifying code. In RSpec, rather than speaking in terms of tests, you talk about *specifications*, expressed in terms of executable examples.

For example, to specify the requirement that a Frequent Flyer should initially have a Bronze Frequent Flyer status, you could write the following specification:

```
describe FrequentFlyer do
  it 'should initially have Bronze status' do
    frequentFlyer = FrequentFlyer.new
    expect(frequentFlyer.status).to eq('BRONZE')
  end
end
```

You're talking about the Frequent Flyer domain object.

What should it do or have?

Create a new Frequent Flyer object.

You expect the status to be Bronze.

This descriptive, example-based approach steers away from mentioning the word "test" at all; it's more in line with the BDD philosophy of living documentation and executable specifications. This can be a great help in getting developers into the BDD mindset of writing specifications rather than tests.

RSpec provides many features that make it easier to structure these executable specifications in a sensible manner. For example, you can group requirements into different contexts to give more background to each set of granular specifications:

```
describe FrequentFlyer do
  context 'when the frequent flyer account is first created' do
    it 'should initially have Bronze status' do
      frequentFlyer = FrequentFlyer.new
      expect(frequentFlyer.status).to eq('BRONZE')
    end
  end

  context 'when a new member starts to fly with Flying High' do
    it 'should earn points for each flight' do
      frequentFlyer = FrequentFlyer.new

      frequentFlyer.earn_status_points(100)
```

❶ **Given...**

Group requirements by context.

Contexts can contain several related requirements.